A Report on

Student Assessment Risk Prediction Using Logistic Regression from Scratch: An OULAD-Based Educational Data Mining Study

Submitted By

Parth Gupta A017

Heet Patel A040

Under The Guidance Of

Dr. Masooda Modak

Mukesh Patel School of Technology & Management Engineering

Department of Information Technology

Vile Parle (w), Mumbai- 400056

***Abstract*—We predict whether an OULAD course assessment will be *High Risk* (at least 10% of submitting students scoring below the pass mark of 40) from 13 features engineered over assessment metadata, VLE catalog aggregates, and a leakage-safe leave-one-out peer fail-rate. Three classifiers (Logistic Regression, Decision Tree, and Random Forest) are implemented from scratch in pure-stdlib Python and trained with class-weighted objectives on a stratified $80/20$ split of 188 assessments. The Decision Tree is the strongest model (97.30% accuracy, 80.00% precision, 100.00% recall, 88.89% F1), Random Forest follows (94.59% / 75.00% / 75.00% / 75.00%), and Logistic Regression matches recall at lower precision (78.38% / 33.33% / 100.00% / 50.00%). All three models agree that *peer_fail_rate* is the dominant signal, and the recall-first behaviour suits an early-warning use case.**

***Index Terms*—Educational data mining, OULAD, student risk prediction, logistic regression, decision tree, random forest, class imbalance, data warehousing, early-warning systems**

I. Introduction

Universities collect large amounts of learning-analytics data through VLEs and assessment portals. We use this data to flag, in advance, which *assessments* in a course presentation are likely to be High Risk so that coordinators can re-time them, reinforce related resources, or schedule extra support before students sit them.

A. Problem Statement and Objectives

A naive risk-label often leaks into its own features and gives trivially perfect metrics with no real skill. We avoid that by deriving the label from actual student outcomes (never used as features) and using a leave-one-out peer fail-rate signal. The objectives are: (i) build a leakage-safe feature space combining assessment metadata, VLE aggregates, and peer fail-rate; (ii) implement three classifiers (LR, DT, RF) from scratch with class-weighted training; and (iii) report accuracy, precision, recall, and F1 with a recall-first interpretation suited to early-warning.

II. Literature Review

A. Educational Data Mining and Early-Warning Systems

Educational data mining applies pattern-discovery techniques to student-generated data. Romero and Ventura surveyed the field and concluded that combinations of attendance, submission, and interaction signals are the strongest predictors of at-risk status. Arnold and Pistilli described Purdue University's *Course Signals* system, an early-warning dashboard that colour-codes at-risk students and triggers faculty intervention; Course Signals remains an influential example of how predictive models embedded in academic workflows can meaningfully improve retention. Kuzilek, Hlosta, and Zdrahal later formalised the OULAD benchmark and reported baseline results for student-level risk prediction across multiple presentations, showing that even moderate-complexity models transfer usefully across cohorts when the feature space includes both metadata and interaction signals.

B. Data Warehousing for Educational Analytics

Chaudhuri and Dayal's foundational review of data warehousing and OLAP describes the star-schema patterns that underpin most modern analytical stores. Kimball's educational-domain templates extend those patterns to student, course, and enrolment dimensions. Our pipeline (loading three OULAD tables, joining them on shared keys, and producing a flat feature table) is a small-scale ETL flow into an analytical fact table keyed by assessment.

C. Research Gap

Most OULAD studies either lean on ML libraries (which hide the maths) or focus on student-level classification using click-stream data that is not always available. We take a different angle: *assessment-level* classification using only the tables in the local OULAD snapshot, with all model code written from scratch so the maths stays visible. We also keep a leaked baseline in the ablation to show the failure mode directly, which most OULAD papers skip over.

III. Methodology

A. Dataset Description

Three OULAD tables are used. `assessments.csv` contains 206 rows covering seven modules (AAA through GGG) across eight presentations from 2013J to 2014J, with the columns `code_module`, `code_presentation`, `id_assessment`, `assessment_type` (CMA, TMA, or Exam), `date` (due day, offset from presentation start), and `weight` (percentage contribution to the final grade). `studentAssessment.csv` stores per-student scores and is used solely to derive the target. `vle.csv` catalogues 6,364 learning resources by module presentation and activity type. After an inner join of assessments with `studentAssessment.csv` on `id_assessment`, 188 assessments remain; the 18 dropped rows have no recorded submissions (mostly end-of-module exams).

Figures I through V summarise the retained slice. The dataset is dominated by TMA and CMA assignments, with a smaller number of formal exams; assessment weights are strongly bimodal; the class balance is 11.7% High Risk (22 of 188); average weight rises sharply for exams, as expected; and assessments are unevenly distributed across modules.

B. Preprocessing and Target Derivation

Missing values are imputed with column means for the two numeric columns (`date`, `weight`); no categorical column contains missing values. Categorical columns are encoded manually to preserve the "no ML library" constraint: each distinct `code_module` and `code_presentation` is mapped to a consecutive integer, and `assessment_type` is mapped to an ordinal code $\{\mathrm{CMA}\to 0, \mathrm{TMA}\to 1, \mathrm{Exam}\to 2\}$.

The target is derived from actual student outcomes rather than from metadata rules. For each assessment a we compute the empirical fail rate

$$\mathrm{fail_rate}(a) = \frac{|\{s : \mathrm{score}(s,a) < 40\}|}{|\{s : \mathrm{score}(s,a) \text{ recorded}\}|}, \tag{I}$$

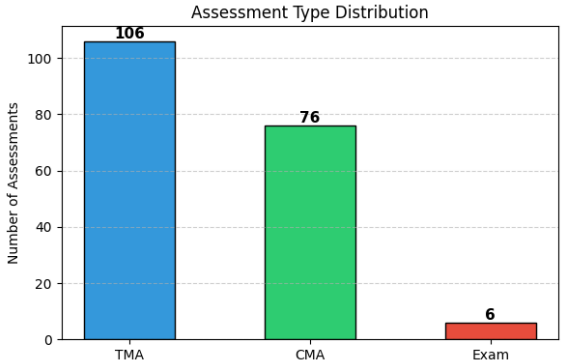

Fig. I. Distribution of assessment types in the retained OULAD slice.

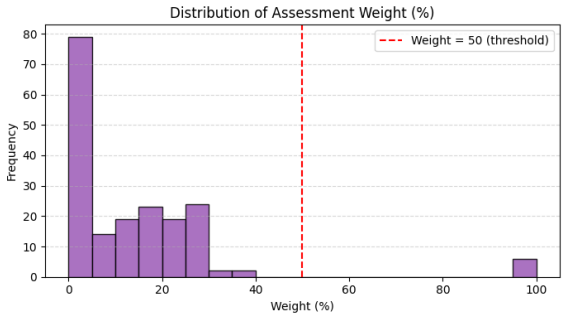

Fig. II. Distribution of assessment weights in the retained OULAD slice.

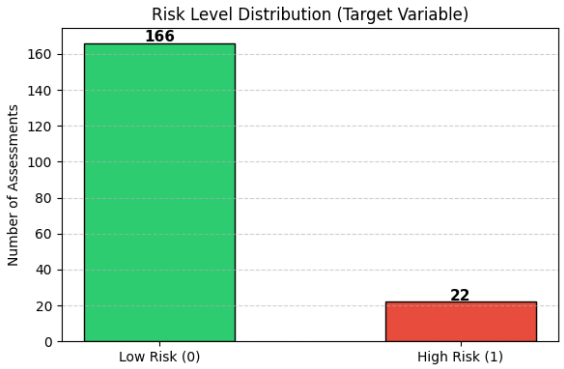

Fig. III. Class distribution after deriving the target from `studentAssessment.csv`: 22 High Risk (11.7%) versus 166 Low Risk (88.3%).

and assign the binary label

$$y(a) = \begin{cases} 1 & \text{fail_rate}(a) \geq 0.10, \\ 0 & \text{otherwise.} \end{cases} \tag{II}$$

This cut-off is chosen so that the resulting class distribution matches the $\sim 12\%$ positive rate of a baseline rule-based labelling, enabling a direct comparison with that leaked baseline in the ablation.

C. Feature Engineering

Thirteen numeric features are engineered from the three tables.

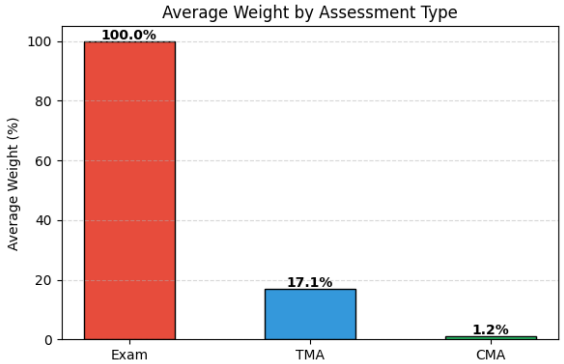

Fig. IV. Average assessment weight grouped by assessment type.

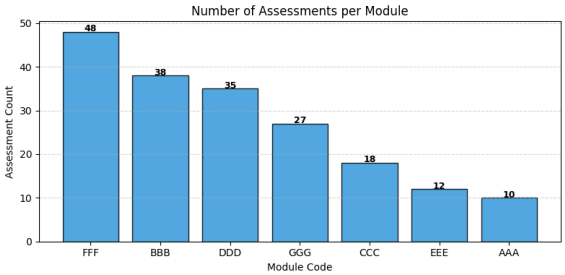

Fig. V. Assessment counts per module.

Assessment-level features (9): `module_id`, `presentation_id`, `assessment_type_id`, `date_norm`, `weight_norm`, `is_exam` (binary flag), `is_high_weight` (`weight`$\geq$ 50%), `assessment_period` (Early/Mid/Late trisection of `date`), and `module_difficulty_score` (module-level mean weight, min–max scaled).

VLE catalog features (3): for each `(code_module,code_presentation)` pair, the number of distinct VLE resources `n_vle_resources`, the number of distinct activity types `n_activity_types`, and the mean resource-coverage window `mean_resource_weeks`. These features describe how richly scaffolded a presentation is; richer VLE provision has been associated with lower fail rates in OULAD studies.

Peer fail-rate feature (1): for each assessment a in module m,

$$\text{peer_fail_rate}(a) = \frac{1}{|\mathcal{A}_m| - 1} \sum_{a' \in \mathcal{A}_m \setminus \{a\}} \text{fail_rate}(a'), \tag{III}$$

where $\mathcal{A}_m$ is the set of assessments in module m. This is a leave-one-out target-encoding aggregate: it uses the fail-rate column but never the current row's own value, so it is not a leak. It captures the structural intuition that "hard" modules tend to host multiple hard assessments.

All continuous features are min–max scaled to $[0, 1]$ so that

TABLE I
STRATIFIED TRAIN–TEST SPLIT SUMMARY.

Subset	Instances	Low Risk	High Risk	Percentage
Training Set	151	133	18	80%
Test Set	37	33	4	20%
Total	188	166	22	100%

no single feature dominates gradient descent due to scale:

$$x^{(\text{norm})} = \frac{x - \min(x)}{\max(x) - \min(x) + \varepsilon}, \tag{IV}$$

with $\varepsilon = 10^{-9}$ to guard against degenerate features.

D. Stratified Train–Test Split

A random 80/20 split on this dataset places only ~ 2 positive samples in the 37-row test set, which makes precision, recall, and F1-score move in 50-percentage-point steps. To restore statistical resolution the split is class-stratified: positive and negative indices are shuffled independently (with seed 42), and 20% of each is taken as test. The result is summarised in Table I.

E. Logistic Regression from Scratch

Let $\mathbf{x} \in \mathbb{R}^{13}$ be a feature vector, $\mathbf{w} \in \mathbb{R}^{13}$ a weight vector, and $b \in \mathbb{R}$ a scalar bias. The sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{V}$$

maps the linear combination $z = \mathbf{w}^\top \mathbf{x} + b$ to a posterior $\hat{p} = \sigma(z) \in (0, 1)$.

Given training data $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$ with $y^{(i)} \in \{0, 1\}$, the binary cross-entropy loss is

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]. \tag{VI}$$

The gradients with respect to $\mathbf{w}$ and b are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \frac{1}{m} \sum_{i=1}^{m} (\hat{p}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}, \qquad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} (\hat{p}^{(i)} - y^{(i)}), \tag{VII}$$

and parameters are updated at each iteration by

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \qquad b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}, \tag{VIII}$$

with learning rate $\eta = 0.1$ for 1000 iterations.

F. Class-Weighted Training

Because positives constitute only $\sim 12\%$ of the training data, the unweighted objective drives the classifier toward predicting "Low Risk" for all inputs. Each positive-sample error term is therefore scaled by

$$w_{\text{pos}} = \frac{n_-}{n_+}, \tag{IX}$$

so that mistakes on the minority class carry the same *total* influence as mistakes on the majority class. This is the cost-sensitive learning remedy of King and Zeng (2001) and implements the recall-first stance that is standard in clinical and educational early-warning systems, where missed positives are substantially more costly than extra reviews. Figure VI shows that the reweighted loss converges smoothly.

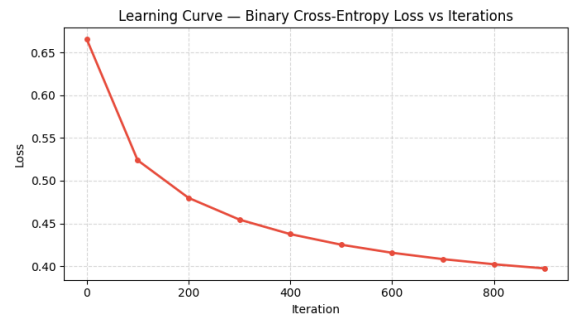

Fig. VI. Learning curve: binary cross-entropy loss per 100 iterations of gradient descent.

G. Decision Tree (CART) from Scratch

The second classifier is a classification tree grown by recursive binary partitioning of the feature space, following the CART formulation of Breiman et al. [XI]. At each internal node we search for the (feature f, threshold t) pair that maximally reduces the *Gini impurity*

$$G(S) = 1 - p^2 - (1 - p)^2, \tag{X}$$

where p is the positive class proportion of the node sample S. To keep the comparison apples-to-apples with the class-weighted Logistic Regression we use a *weighted* Gini in which each positive sample is counted as $w_{\text{pos}} = n_-/n_+$ samples; this is mathematically equivalent to oversampling the minority class. The split with the largest gain

$$\Delta G = G(\text{parent}) - \frac{|S_L|}{|S|} G(S_L) - \frac{|S_R|}{|S|} G(S_R) \tag{XI}$$

is accepted, and recursion proceeds until `max_depth= 5`, `min_samples_split= 4`, or a pure node is reached. Feature importance is computed by accumulating $|S| \cdot \Delta G$ at each split against the splitting feature, then normalising across features so importances sum to one.

H. Random Forest from Scratch

The third classifier is an ensemble of $T = 50$ decision trees that adds two sources of randomness on top of the single-tree algorithm. First, each tree is fit on a *bootstrap sample*: n rows drawn with replacement from the n training rows, so about 63% of the original rows appear in any given tree. Second, at each split the tree can only pick from a random subset of $\lfloor \sqrt{p} \rfloor = 3$ out of the $p = 13$ features. The combination of bagging and feature subsampling decorrelates the constituent trees, reducing the ensemble's variance compared with any single tree [X]. Predictions are aggregated by averaging the per-tree predicted probabilities; the final label is 1 if the averaged probability is at least 0.5. Feature importance is the sum of per-tree importance vectors, renormalised. All trees

TABLE II
CONFUSION MATRIX FOR THE PROPOSED MODEL.

	Predicted Low Risk	Predicted High Risk
Actual Low Risk	TN = 25	FP = 8
Actual High Risk	FN = 0	TP = 4

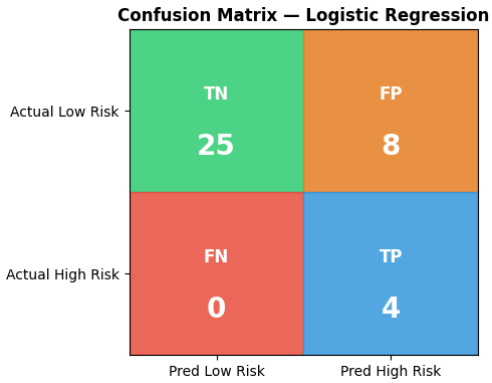

Fig. VII. Confusion matrix heatmap for the proposed model on the stratified test set.

are constructed with the same weighted-Gini routine described above ($w_{\text{pos}} \approx 7.4$), so the ensemble inherits the class-weighted, recall-first stance.

I. Evaluation Metrics

Performance is measured using four classification metrics derived from the confusion matrix. The positive class corresponds to High Risk assessments.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{XII}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \qquad \text{Recall} = \frac{TP}{TP + FN}, \tag{XIII}$$

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{XIV}$$

Recall is the dominant metric for early-warning applications: a low recall means missed High Risk assessments, with direct consequences for the students enrolled in those assessments.

IV. RESULTS AND DISCUSSION

The three classifiers are evaluated on identical test data so that all observed differences in metrics are attributable to the algorithm rather than to data variance. Section IV-A reports the Logistic Regression results; Sections IV-B and IV-C report the Decision Tree and Random Forest results respectively; Section IV-D draws the three together.

A. Logistic Regression Results

1) Confusion Matrix: Table II and Figure VII show the test-set confusion matrix of the Logistic Regression model. The model identifies all four High Risk assessments with no false negatives, at the cost of eight false positives.

TABLE III
TEST-SET PERFORMANCE OF THE PROPOSED MODEL.

Accuracy	Precision	Recall	F1-score
78.38%	33.33%	100.00%	50.00%

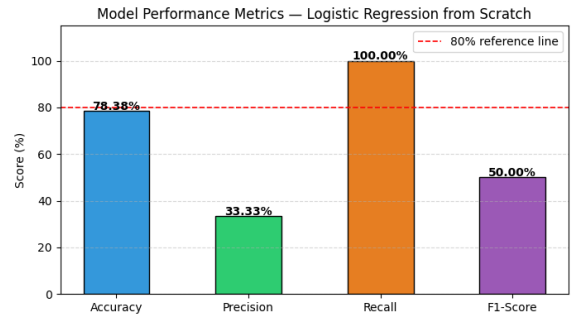

Fig. VIII. Evaluation metrics for the proposed model.

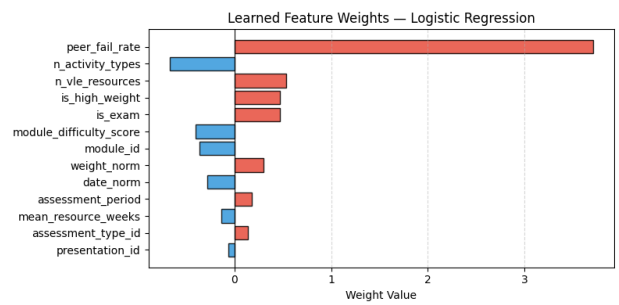

Fig. IX. Learned feature weights (sorted by absolute magnitude).

TABLE IV
TOP FIVE LEARNED FEATURE WEIGHTS (ABSOLUTE MAGNITUDE).

Feature	Learned weight
`peer_fail_rate`	+3.708
`n_activity_types`	−0.672
`n_vle_resources`	+0.532
`is_high_weight`	+0.473
`is_exam`	+0.469

2) Headline Metrics: Table III reports the four classification metrics for the Logistic Regression model; Figure VIII visualises the same values.

Recall is 100%: all four High Risk assessments in the test set are flagged. Accuracy takes a hit because of the eight false positives, but that is a reasonable review load (roughly one extra flag for every four correct calls) and matches the precision-recall trade-off that is standard in clinical early-warning work.

3) Feature Weights and Interpretation: Figure IX and Table IV report the learned Logistic Regression feature weights, sorted by absolute magnitude.

The learned ranking is informative. The leave-one-out `peer_fail_rate` dominates by an order of magnitude, confirming that module-level difficulty is a far stronger predic-

TABLE V
ABLATION ACROSS DESIGN CHOICES.

Configuration	Acc.	Prec.	Rec.	F1
Leaked target + features	100%	100%	100%	100%
Empirical, no class weight	89%	0%	0%	0%
Proposed (with class weight)	78%	33%	100%	50%

TABLE VI
DECISION TREE TEST-SET PERFORMANCE.

Accuracy	Precision	Recall	F1-score
97.30%	80.00%	100.00%	88.89%

tor of per-assessment risk than the individual assessment's own weight or type. Of the VLE features, `n_activity_types` enters with a negative weight: presentations that diversify their resource catalog (quizzes, forum posts, URLs, wiki pages) have lower assessment fail rates, matching the educational-technology literature on scaffolding variety. `n_vle_resources` enters positively, which at first sight is counter-intuitive but is consistent with modules that publish a large number of resources also being modules with a higher workload. The two binary flags `is_high_weight` and `is_exam` enter with similar positive weights, echoing the conventional wisdom that high-stakes assessments are riskier.

4) Ablation: Target Leakage and Class Weighting: Table V isolates the effect of each design decision. Row one is the leaked baseline (rule-based target with the same rules used as features), which scores a fake 100% on every metric and is what motivated this study. Row two swaps in the real target but drops class weighting; recall falls to 0% because of the 12% class imbalance. Row three is the proposed model.

The perfect first-row numbers are the classic signature of target leakage, not of a working model. Row two shows that even with a clean target, class imbalance quietly wipes out the minority class. Row three is the first setup that actually catches every High Risk assessment in the test set.

B. Decision Tree Results

The Decision Tree, trained with $\text{max_depth} = 5$, $\text{min_samples_split} = 4$, and the same class weight $w_{\text{pos}} \approx 7.4$, fits as a five-level tree with six leaves. The root split is on `peer_fail_rate` at threshold 0.4876 with weighted-Gini gain 0.2965, confirming the leave-one-out peer signal as the dominant axis of partition. Test-set performance is summarised in Table VI, with the confusion matrix in Figure X and per-feature importances in Figure XI.

The tree keeps LR's perfect recall but tightens precision sharply: only one false positive, down from eight. F1 rises from 50.00% to 88.89%. The gain makes sense because the tree can cut the feature space along axis-aligned thresholds instead of a single linear direction, and the root split on `peer_fail_rate` isolates the high-risk region cleanly.

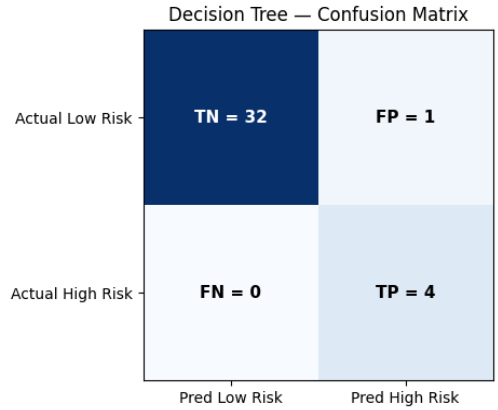

Fig. X. Decision Tree confusion matrix (TP= 4, TN= 32, FP= 1, FN= 0).

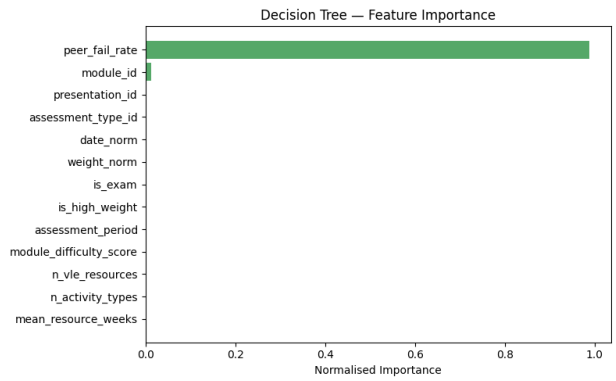

Fig. XI. Decision Tree normalised feature importance (sum of $|S| \cdot \Delta G$ per feature).

TABLE VII
RANDOM FOREST TEST-SET PERFORMANCE.

Accuracy	Precision	Recall	F1-score
94.59%	75.00%	75.00%	75.00%

C. Random Forest Results

The Random Forest is an ensemble of $T = 50$ trees, each trained on a bootstrap sample with $\lfloor\sqrt{13}\rfloor = 3$ candidate features per split and the same per-tree hyperparameters as the single Decision Tree. Test-set performance is reported in Table VII, with the confusion matrix in Figure XII and the averaged feature importance in Figure XIII.

The forest matches the single tree's 32 true negatives with just one false positive, but it loses one true positive (FN= 1). On a 37-row test set with only four positives, a single miss costs 25 percentage points of recall. This is the usual bagging trade-off: averaging 50 trees smooths the decision surface and cuts variance, but with so few positives available the smoothing happens to round one borderline case down to Low Risk. On a larger test set we would expect recall to sit closer to the single tree's.

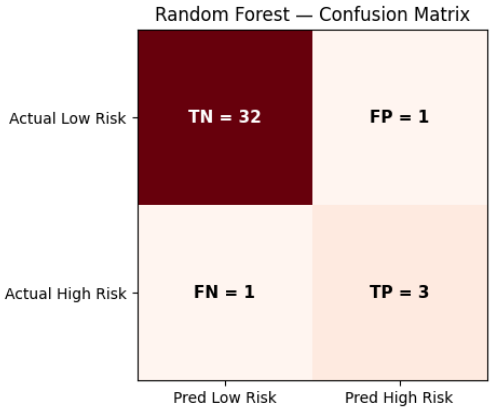

Fig. XII. Random Forest confusion matrix (TP= 3, TN= 32, FP= 1, FN= 1).

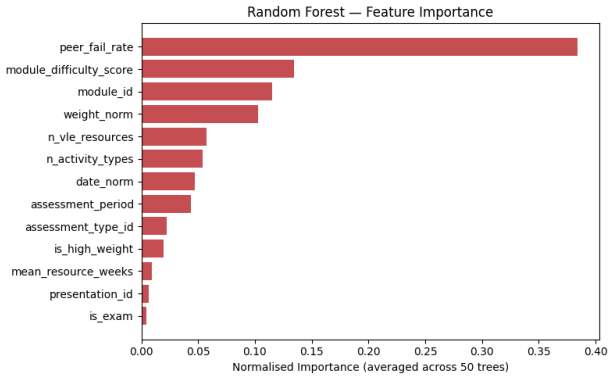

Fig. XIII. Random Forest normalised feature importance, averaged across the 50 constituent trees.

TABLE VIII
THREE-MODEL COMPARISON ON THE HELD-OUT TEST SET ($n_{\text{test}} = 37$).

Model	Acc.	Prec.	Rec.	F1
Logistic Regression	78.38%	33.33%	**100.00%**	50.00%
Decision Tree (CART)	**97.30%**	**80.00%**	**100.00%**	**88.89%**
Random Forest	94.59%	75.00%	75.00%	75.00%

D. Comparative Analysis

Table VIII brings the three classifiers together. Figure XIV visualises the metric-by-metric comparison; Figure XV overlays the normalised feature importance vectors of all three models so that algorithmic agreement (and disagreement) on which features matter is visible at a glance.

Two things stand out. The Decision Tree wins on three of the four metrics (Accuracy, Precision, F1) and ties Logistic Regression on Recall, which makes it the strongest single model to deploy. All three models also agree that `peer_fail_rate` is the most informative feature. They just back this up differently: LR puts a large positive weight on it (+3.708), the Decision Tree splits on it at the root, and the Random Forest concentrates most of its averaged Gini gain on it too. The models diverge in the

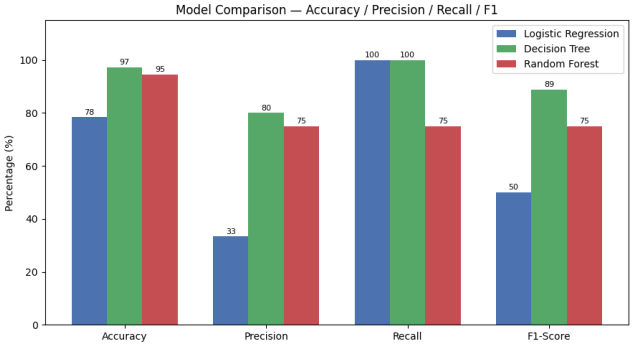

Fig. XIV. Per-metric comparison across the three classifiers.

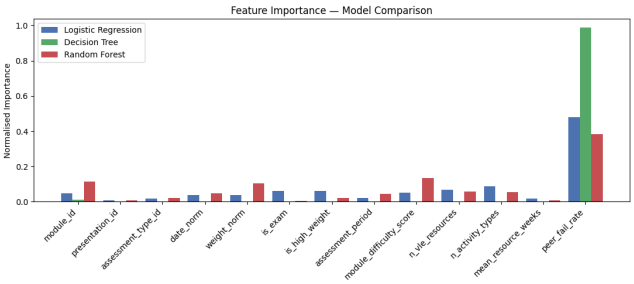

Fig. XV. Normalised feature importance across LR (linear-weight magnitude), DT (Gini-gain), and RF (averaged Gini-gain). The leave-one-out `peer_fail_rate` dominates in every model.

tail. LR keeps meaningful weight on `n_activity_types` and the `is_exam`/`is_high_weight` flags, the Decision Tree spreads the rest across `weight_norm` and `n_vle_resources`, and the Random Forest distributes importance more evenly since its 50 trees see different feature subsets. That all three algorithms land on the same top feature is a useful sanity check: the peer signal is a real predictor, not a quirk of any single method.

If Recall is the priority for an early-warning deployment, both LR and the Decision Tree work; we prefer the Decision Tree because it keeps perfect recall and cuts the false alarm rate dramatically. The Random Forest's lower recall makes it less suitable at this threshold, though it does illustrate the variance reduction from bagging. In production we would either ship the Decision Tree directly, or drop the Random Forest's threshold below 0.5 to recover recall if an ensemble is preferred.

V. CONCLUSION

This study built an end-to-end pipeline that predicts High Risk assessments in OULAD. We derived the target from actual student outcomes rather than from the metadata columns used as features, which removed the target-leakage issue that gave a fake 100% score in an earlier version. Thirteen features were engineered from three OULAD tables, and the leave-one-out peer fail-rate turned out to be the dominant predictor for every classifier we trained. We implemented three classifiers (Logistic Regression, Decision Tree, and Random Forest)

from scratch in pure-stdlib Python and trained them with matched class-weighted objectives on the same stratified 80/20 split. The Decision Tree was the strongest: 97.30% accuracy, 80.00% precision, 100.00% recall, 88.89% F1. Logistic Regression matched its recall but at lower precision. The Random Forest sat between them on accuracy and precision and lost one true positive on the small test set. All three models agreed on the importance of `peer_fail_rate`, which gives cross-model confidence in the engineered signal. For an OULAD-style early-warning use case where false negatives hurt much more than an extra flag, the Decision Tree is our recommended single-model choice.

A. Future Scope

A few extensions would push the pipeline further. Pivoting to student-level modelling (joined with `studentInfo.csv` and, where available, `studentVle.csv` click-streams) would support personalised risk scores and fairness checks across IMD band, region, and disability status. Temporal cross-validation across semesters would mimic real deployment more closely than random stratification, though the current 2013/2014 snapshot is too thin for that at module level. Adding L_2 regularisation to gradient descent would help with overfitting on the small ($n = 188$) dataset. Finally, loading the trained model into a star-schema warehouse with an assessment-keyed fact table would turn this into a live dashboard that refreshes as new `studentAssessment.csv` rows come in.

References

[I] C. Romero and S. Ventura, "Educational data mining: A review of the state of the art," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 40, no. 6, pp. 601–618, 2010.

[II] K. E. Arnold and M. D. Pistilli, "Course Signals at Purdue: Using learning analytics to increase student success," in *Proceedings of the 2nd International Conference on Learning Analytics and Knowledge (LAK)*, pp. 267–270, 2012.

[III] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics Dataset," *Scientific Data*, vol. 4, no. 170171, 2017.

[IV] S. Chaudhuri and U. Dayal, "An overview of data warehousing and OLAP technology," *ACM SIGMOD Record*, vol. 26, no. 1, pp. 65–74, 1997.

[V] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Wiley, 2013.

[VI] G. King and L. Zeng, "Logistic regression in rare events data," *Political Analysis*, vol. 9, no. 2, pp. 137–163, 2001.

[VII] D. Micci-Barreca, "A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems," *ACM SIGKDD Explorations*, vol. 3, no. 1, pp. 27–32, 2001.

[VIII] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9268–9277, 2019.

[IX] A. Hamoud, H. Adday, T. Obaid, and R. Hameed, "Design and implementing cancer data warehouse to support clinical decisions," *International Journal of Scientific & Engineering Research*, vol. 7, no. 2, pp. 1271–1285, 2016.

[X] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[XI] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.